

JEGYZŐKÖNYV

Adatbázis rendszerek II.

Féléves feladat – PL/SQL

Készítette: Csubák Bálint Miklós

Neptunkód: LVSS9I

Dátum: 2026.05.05

AI: 5-10%

Tartalom

A FELADAT LEÍRÁSA	3
A táblát létrehozó sql parancsok	4
A táblát feltöltő sql parancsok	4
1.) Tárolt eljárás adatok felvitelére - A rekord minden mezőjébe vigyen be adatot!	5
2.)Tárolt eljárás adatok módosítására	6
3.)Tárolt eljárás adatok törlésére	7
4.)Tárolt függvény adott rekord mezőinek lekérdezésére	8
5.)Tárolt függvény adott feltételű rekordok aggregált értékének lekérdezésére	9
7.)Trigger készítése módosítási események naplózására	10
8.)Trigger készítése kulcs érték automatikus megadására	11
9.)Trigger készítése a módosítások kontrollálására.....	12
TESZTELŐ KÓDOK.....	13
Mélytörlés	14

A FELADAT LEÍRÁSA

A következő projektfeladat az Adatbázis rendszerek II. egyetemi tantárgy féléves munka keretein belül készült, célja a szerver oldali PL/SQL programozás gyakorlati alkalmazása az Oracle (Apex) adatbázis-kezelő környezetben. A feladat egy korábban kiépített relációs adatbázist használ fel, melyből egy táblával kell műveleteket végezni, mely szerint választásom az ügyféladatokat tartalmazó UGYFEL táblára esett, hiszen ez sokféle adattípust tartalmaz, és minden feladatpont megvalósítható benne. A feladatkiírásnak megfelelően létrehoztam egy műveletnaplózásért felelős NAPLO táblát is, mely az adatbázisban elvégzett műveleteket tartja nyilván. A megoldásokban implementálni kellett a szabványos SQL parancsokat, az implicit és explicit kurzorokat, valamint a hibák biztonságos lefutását garantáló kivételkezelést. A megvalósítandó adatbázis-oldali funkciók az alábbi kötelező lépésekből tevődnek össze:

Alapvető adatkezelés (CRUD műveletek tárolt eljárásokkal): Olyan PL/SQL eljárásoknak az elkészítése, amelyek garantálják az adatok biztonságos és helyes felvitelét, az adatok módosítását, valamint törlését egyedi azonosító vagy egy tulajdonság alapján. Adatlekérdezés és számítások (Tárolt függvények): Függvények fejlesztése az adatok kinyerésére. Készült paraméteres és paraméter nélküli lekérdezés a rekordok mezőinek megjelenítésére (külön lekezelve a nem létező rekordokból adódó kivételeket), illetve egy aggregáló függvény, amely megadott feltételek mentén összesített értékeket számít ki az adatbázisból. Kódok egységbe zárása: Egy PL/SQL csomag (Package) létrehozása, amely logikailag összefogja a táblához tartozó, de legalább két darab megírt tárolt alprogramot. Automatizálás és adatvédelem (Triggererek): Az adatbázis önműködésének és konzisztenciájának biztosítása érdekében három trigger implementálása volt az elvárás. Ezek feladata az elsődleges kulcs értékének automatikus generálása beszúráskor, a táblát érintő módosítási események (beszúrás, frissítés, törlés) részletes naplózása, valamint a módosítások szigorú ellenőrzése és kontrollálása a megadott szabályok szerint.

A táblát létrehozó sql parancsok

A projekt alapját a relációs adatbázis sémájának megtervezése adta. A JDBC feladatban kiépített adatbázisból kiválasztottam a számomra megfelelő táblát, majd átalakítottam úgy, hogy az APEX is könnyedén tudja kezelni. A táblában szerepelnek kötelező megkötések, többféle típus és alapértelmezett értékek. A választott UGYFEL tábla egy étterem regisztrált ügyfeleit tárolja, az ügyfelek legfontosabb adataival.

```
CREATE TABLE UGYFEL (  
    id NUMBER PRIMARY KEY,  
    nev VARCHAR2(50) NOT NULL,  
    cim VARCHAR2(100) NOT NULL,  
    mobil VARCHAR2(12) NOT NULL,  
    regisztralt DATE DEFAULT SYSDATE NOT NULL,  
    szallitas NUMBER NOT NULL  
);
```

A táblát feltöltő sql parancsok

A szerkezet kialakítása után szükség volt egy induló adatkészlet betöltésére, hogy a későbbi PL/SQL eljárásokat tesztelni lehessen. Ezeket véletlenszerűen generált, nem valóságos adatokkal töltöttem fel. A kód végén kiadott COMMIT utasítás segítségével véglegesítem a műveleteket, hogy fizikailag a lemezre kerüljön.

```
BEGIN  
    INSERT INTO UGYFEL (id, nev, cim, mobil, regisztralt, szallitas)  
    VALUES (1, 'Kovács János', '1032 Budapest, Bécsi út 45.', '301234567', TO_DATE('2000-01-01', 'YYYY-MM-DD'), 20);  
  
    INSERT INTO UGYFEL (id, nev, cim, mobil, regisztralt, szallitas)  
    VALUES (2, 'Nagy Erzsébet', '4024 Debrecen, Piac utca 10.', '309876543', TO_DATE('2000-01-01', 'YYYY-MM-DD'), 20);  
  
    INSERT INTO UGYFEL (id, nev, cim, mobil, regisztralt, szallitas)  
    VALUES (3, 'Szabó Péter', '6720 Szeged, Tisza Lajos krt. 5.', '305554433', TO_DATE('2000-01-01', 'YYYY-MM-DD'), 20);  
  
    INSERT INTO UGYFEL (id, nev, cim, mobil, regisztralt, szallitas)  
    VALUES (4, 'Tóth Anita', '8000 Székesfehérvár, Fő tér 1.', '302221188', TO_DATE('2000-01-01', 'YYYY-MM-DD'), 20);  
  
    COMMIT;  
END;
```

1.) Tárolt eljárás adatok felvitelére - A rekord minden mezőjébe vigyen be adatot!

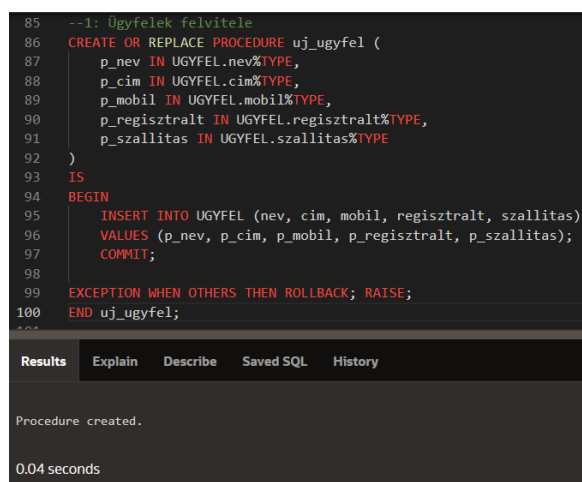
A feladat célja egy olyan szerver oldali tárolt eljárás (PROCEDURE) létrehozása volt, amely biztonságos, paraméterezett és kontrollált módon teszi lehetővé új ügyfelek rögzítését az adatbázisban. A tervezés során úgy alakítottam ki az eljárást, hogy mindig igazodjanak a beérkező adatok típusai az UGYFEL tábla mezőtípusaihoz. Ezáltal dinamikusabbá téve, és lecsökkentve a jövőbeni hibák számát. Az egyedi azonosító (ID) generálását egy későbbi trigger végzi majd, így az eljárás bemeneti paraméterei közé az ID nem került be.

Dinamikus típusmegadást (%TYPE) használtam a konkrét típusok (pl. VARCHAR2) helyett, hogy ne kelljen feleslegesen ugyanazokat beállítani, amiket már a táblalétrehozáskor beállítottam, és, hogy a tábla esetleges módosítása esetén csökkentsem a hibák lehetőségét. Minden szükséges bemenő adatot eltároltam p_... néven.

Ezután megkísérlem beilleszteni a táblába a beérkező adatokat a megfelelő mezőkbe, mely beillesztést véglegesítek, amennyiben nem merült fel hiba. Hiba esetén kivételkezelést használok, mely minden váratlan hiba esetén lefut. Ekkor a ROLLBACK utasítás visszavonja a műveleteket, azaz nem véglegesíti azokat, hogy megakadályozza a hibás vagy hiányos adatok bekerülését a táblába. A RAISE parancs pedig kiírja a hibát.

```
--1: Ügyfelek felvitele
CREATE OR REPLACE PROCEDURE uj_ugyfel (
    p_nev IN UGYFEL.nev%TYPE,
    p_cim IN UGYFEL.cim%TYPE,
    p_mobil IN UGYFEL.mobil%TYPE,
    p_regisztralt IN UGYFEL.regisztralt%TYPE,
    p_szállitas IN UGYFEL.szállitas%TYPE
)
IS
BEGIN
    INSERT INTO UGYFEL (nev, cim, mobil, regisztralt, szállitas)
    VALUES (p_nev, p_cim, p_mobil, p_regisztralt, p_szállitas);
    COMMIT;

EXCEPTION WHEN OTHERS THEN ROLLBACK; RAISE;
END uj_ugyfel;
```



```
85 --1: Ügyfelek felvitele
86 CREATE OR REPLACE PROCEDURE uj_ugyfel (
87     p_nev IN UGYFEL.nev%TYPE,
88     p_cim IN UGYFEL.cim%TYPE,
89     p_mobil IN UGYFEL.mobil%TYPE,
90     p_regisztralt IN UGYFEL.regisztralt%TYPE,
91     p_szállitas IN UGYFEL.szállitas%TYPE
92 )
93 IS
94 BEGIN
95     INSERT INTO UGYFEL (nev, cim, mobil, regisztralt, szállitas)
96     VALUES (p_nev, p_cim, p_mobil, p_regisztralt, p_szállitas);
97     COMMIT;
98
99     EXCEPTION WHEN OTHERS THEN ROLLBACK; RAISE;
100 END uj_ugyfel;
```

Results	Explain	Describe	Saved SQL	History
Procedure created.				
0.04 seconds				

2.)Tárolt eljárás adatok módosítására

A feladat elvárása egy olyan tárolt eljárás megvalósítása volt, amely képes egy meglévő rekord bizonyos adatainak (jelen esetben az ügyfél mobilszámának) módosítására, az egyedi azonosító (ID) alapján. Az ID-t sosem módosítjuk, így az eljárás paraméterként kapja csak meg az ID-t, viszont kizárólag más adatokat fog módosítani.

Ugyanúgy dinamikus típusmegadást használtam, minden bemenő adatra. Készítettem egy előzetes vizsgáló lekérdezést, hogy még a módosítások előtt leellenőrizzem, hogy létezik-e a megadott ID a táblában. Ezt úgy csináltam, hogy megszámláltam, hogy hányszor van benne a táblában az adott ID. Mivel normális esetben 1-nél több eredmény nem lehet, így vagy 0, vagy 1 eredményt feltételeztem, melyet betöltöttem az előre létrehozott i nevű változóba.

Ha 0-át kapok, akkor tudom, hogy nincsen a táblában ilyen ID, ezért nem tudnék mit módosítani. Ezt egy hibaüzenettel jelzem, és lezárom az eljárást. Ha 1-et kapok, akkor módosítom a mobilszámot az adott egyedi azonosítóval rendelkező ügyfélnél. Ha mindez sikeres, véglegesítem. Hiba esetén kivételkezelést alkalmaztam most is.

```
--2: Mobilszám módosítása
CREATE OR REPLACE PROCEDURE ugyfel_mobil (
    p_id      IN UGYFEL.id%TYPE,
    p_mobil IN UGYFEL.mobil%TYPE
)
IS
i NUMBER;
BEGIN
    SELECT COUNT(*) INTO i FROM UGYFEL WHERE id = p_id;
    IF i = 0 THEN RAISE_APPLICATION_ERROR(-20000, 'Nincs ilyen ügyfél!'); END IF;
    UPDATE UGYFEL SET mobil = p_mobil WHERE id = p_id;
    COMMIT;

EXCEPTION WHEN OTHERS THEN ROLLBACK; RAISE;
END ugyfel_mobil;
```

```
105  --2: Mobilszám módosítása
106  CREATE OR REPLACE PROCEDURE ugyfel_mobil (
107      p_id      IN UGYFEL.id%TYPE,
108      p_mobil IN UGYFEL.mobil%TYPE
109  )
110  IS
111  i NUMBER;
112  BEGIN
113      SELECT COUNT(*) INTO i FROM UGYFEL WHERE id = p_id;
114      IF i = 0 THEN RAISE_APPLICATION_ERROR(-20000, 'Nincs ilyen ügyfél!'); END IF;
115      UPDATE UGYFEL SET mobil = p_mobil WHERE id = p_id;
116      COMMIT;
117
118  EXCEPTION WHEN OTHERS THEN ROLLBACK; RAISE;
119  END ugyfel_mobil;
120
```

Results	Explain	Describe	Saved SQL	History
Procedure created.				
0.03 seconds				

3.)Tárolt eljárás adatok törlésére

A feladat egy olyan tárolt eljárás megírását írta elő, amellyel törölhetők adatok az adatbázisból, akár egyedi azonosító, akár egy adott tulajdonság alapján. Én mindkét esetet megvalósítottam. Az egyik eljárás a megadott ID alapján törli az ügyfelet, a másik pedig azokat az ügyfeleket törli, melyek túl messze vannak, azaz ahol túl nagy a szállítási idő, melyet az alapján határoz meg, hogy bemenő értékként várom a maximális szállítási időt. Ami annál több, azokat az ügyfeleket törlöm. A feladat megoldása során a feladatkiírásban szereplő implicit kurzort (SQL%ROWCOUNT) is alkalmaztam, mely megmutatja, hogy hány ügyfél lett törölve. Ez azért is hasznos, mert tudom jelezni, ha nem történt törlés. Ezután szintén kivételkezelést is alkalmaztam.

```
--3: Törlés ID szerint
CREATE OR REPLACE PROCEDURE ugyfeltorles_id ( p_id IN UGYFEL.id%TYPE )
IS
  i NUMBER;
BEGIN
  DELETE FROM UGYFEL WHERE id = p_id;
  i := SQL%ROWCOUNT;
  IF i = 0 THEN RAISE_APPLICATION_ERROR(-20000, 'Nincs ilyen ügyfél!'); END IF;
  COMMIT;

EXCEPTION WHEN OTHERS THEN ROLLBACK; RAISE;
END ugyfeltorles_id;

-----

--3: Törlés szállítási idő szerint
CREATE OR REPLACE PROCEDURE ugyfeltorles_szallitas( p_maxido IN UGYFEL.szallitas%TYPE )
IS
  i NUMBER;
BEGIN
  DELETE FROM UGYFEL WHERE szallitas > p_maxido;
  i := SQL%ROWCOUNT;
  IF i = 0 THEN RAISE_APPLICATION_ERROR(-20000, 'Nem történt törlés!'); END IF;
  COMMIT;

EXCEPTION WHEN OTHERS THEN ROLLBACK; RAISE;
END ugyfeltorles_szallitas;
```

```
123 --3: Törlés ID szerint
124 CREATE OR REPLACE PROCEDURE ugyfeltorles_id ( p_id IN UGYFEL.id%TYPE )
125 IS
126   i NUMBER;
127 BEGIN
128   DELETE FROM UGYFEL WHERE id = p_id;
129   i := SQL%ROWCOUNT;
130   IF i = 0 THEN RAISE_APPLICATION_ERROR(-20000, 'Nincs ilyen ügyfél!'); END IF;
131   COMMIT;
132
133 EXCEPTION WHEN OTHERS THEN ROLLBACK; RAISE;
134 END ugyfeltorles_id;
135
136
137
```

Results	Explain	Describe	Saved SQL	History
Procedure created.				
0.02 seconds				

```
138
139 --3: Törlés szállítási idő szerint
140 CREATE OR REPLACE PROCEDURE ugyfeltorles_szallitas( p_maxido IN UGYFEL.szallitas%TYPE )
141 IS
142   i NUMBER;
143 BEGIN
144   DELETE FROM UGYFEL WHERE szallitas > p_maxido;
145   i := SQL%ROWCOUNT;
146   IF i = 0 THEN RAISE_APPLICATION_ERROR(-20000, 'Nem történt törlés!'); END IF;
147   COMMIT;
148
149 EXCEPTION WHEN OTHERS THEN ROLLBACK; RAISE;
150 END ugyfeltorles_szallitas;
151
```

Results	Explain	Describe	Saved SQL	History
Procedure created.				
0.02 seconds				

4.)Tárolt függvény adott rekord mezőinek lekérdezésére

A feladat elvárása szerint olyan tárolt függvényeket (FUNCTION) kellett létrehozni, amelyek képesek adatokat lekérdezni. Két függvényt is készítettem. Az egyik megkeresi és kiírja a megadott id-hoz tartozó ügyfél nevét. A másik pedig kiírja a megadott id-hoz tartozó ügyfél minden adatát. Amennyiben nem találtam adatot, azt kiírtam a feladat szerint.

A lekérdezést egy %ROWTYPE típusú változóval oldottam meg, mely azért jó, mert egész sorokat képes tárolni. Mivel normális esetben mindig csak egy rekordot kell kapjunk, ezért ezt megfelelőnek találtam. Az adatokat összefűztem, melynek megkönnyítése érdekében a dátumot átalakítottam szöveg típusúvá. Továbbá beállítottam a dátumnak a magyar dátumkijelzést, hogy szebb legyen. Itt is kivételkezelést végeztem, melyet kimondottan úgy csináltam, hogy akkor dobjon hibát, ha nincsen ilyen rekord. És ezt ki is írja a terminálra. A függvények teszteléséhez a beépített DUAL virtuális táblát használtam.

```
--4: Név listázása
CREATE OR REPLACE FUNCTION find_nev ( f_id IN UGYFEL.id%TYPE ) RETURN VARCHAR2
IS
f_nev UGYFEL.nev%TYPE;
BEGIN
    SELECT nev INTO f_nev FROM UGYFEL WHERE id = f_id;
    RETURN f_nev;

    EXCEPTION WHEN NO_DATA_FOUND THEN RETURN 'Nincs ilyen rekord!';
END find_nev;
```

```
--4: Ügyféladatok listázása
CREATE OR REPLACE FUNCTION ugyfeladatok ( f_id IN UGYFEL.id%TYPE ) RETURN VARCHAR2
IS
adatok UGYFEL%ROWTYPE;
BEGIN
    SELECT * INTO adatok FROM UGYFEL WHERE id = f_id;
    RETURN
        adatok.nev || ' | ' || ' ||
        adatok.cim || ' | ' || ' ||
        adatok.mobil || ' | ' || ' ||
        TO_CHAR(adatok.regisztralt, 'YYYY.MM.DD') || ' | ' || ' ||
        adatok.szallitas;

    EXCEPTION WHEN NO_DATA_FOUND THEN RETURN 'Nincs ilyen rekord!';
END ugyfeladatok;
```

```
154 --4: Név listázása
155 CREATE OR REPLACE FUNCTION find_nev ( f_id IN UGYFEL.id%TYPE ) RETURN VARCHAR2
156 IS
157 f_nev UGYFEL.nev%TYPE;
158 BEGIN
159     SELECT nev INTO f_nev FROM UGYFEL WHERE id = f_id;
160     RETURN f_nev;
161
162     EXCEPTION WHEN NO_DATA_FOUND THEN RETURN 'Nincs ilyen rekord!';
163 END find_nev;
164
165
166
167
168
169 4. Ügyféladatok listázása
```

Results	Explain	Describe	Saved SQL	History
Function created.				
0.04 seconds				

```
168 --4: Ügyféladatok listázása
169 CREATE OR REPLACE FUNCTION ugyfeladatok ( f_id IN UGYFEL.id%TYPE ) RETURN VARCHAR2
170 IS
171 adatok UGYFEL%ROWTYPE;
172 BEGIN
173     SELECT * INTO adatok FROM UGYFEL WHERE id = f_id;
174     RETURN
175         adatok.nev || ' | ' || ' ||
176         adatok.cim || ' | ' || ' ||
177         adatok.mobil || ' | ' || ' ||
178         TO_CHAR(adatok.regisztralt, 'YYYY.MM.DD') || ' | ' || ' ||
179         adatok.szallitas;
180
181     EXCEPTION WHEN NO_DATA_FOUND THEN RETURN 'Nincs ilyen rekord!';
182 END ugyfeladatok;
```


5.)Tárolt függvény adott feltételű rekordok aggregált értékének lekérdezésére

A feladat egy olyan tárolt függvény megvalósítása volt, amely egy számított értéket ad eredményül. A táblából a szállítási időt választottam. Beérkezik a helység (helységnév és irányítószám is lehet), majd az összes olyan rekordot a táblából, aminek a címe tartalmazza ezt, összegzi a szállítási időket, és kiszámolja az adott településre az átlagos szállítási időt az AVG függvény segítségével.

```
--5: Átlagos szállítási idő településenként
CREATE OR REPLACE FUNCTION szallitasido ( f_helyseg IN VARCHAR2) RETURN NUMBER
IS
atlagido NUMBER;
BEGIN
    SELECT AVG(szallitas) INTO atlagido FROM UGYFEL WHERE cim LIKE '%' || f_helyseg || '%';

    IF atlagido IS NULL THEN RAISE_APPLICATION_ERROR(-20000, 'Nincs találat!'); END IF;
    RETURN atlagido;
END szallitasido;
```

```
186  --5: Átlagos szállítási idő településenként
187  CREATE OR REPLACE FUNCTION szallitasido ( f_helyseg IN VARCHAR2) RETURN NUMBER
188  IS
189  atlagido NUMBER;
190  BEGIN
191      SELECT AVG(szallitas) INTO atlagido FROM UGYFEL WHERE cim LIKE '%' || f_helyseg || '%';
192
193      IF atlagido IS NULL THEN RAISE_APPLICATION_ERROR(-20000, 'Nincs találat!'); END IF;
194      RETURN atlagido;
195  END szallitasido;
196
197
```

Results	Explain	Describe	Saved SQL	History
---------	---------	----------	-----------	---------

Function created.

0.03 seconds

7.)Trigger készítése módosítási események naplózására

A feladat egy olyan eseményvezérlő (trigger) létrehozása volt, amely az UGYFEL táblán végzett műveleteket, mint az adatbeszúrás, módosítás, törlés automatikusan naplózza egy külön erre a célra létrehozott NAPLO táblába. Az EXECUTE IMMEDIATE parancsokat csak azért használtam, hogy ne kelljen külön-külön kiadni a parancsokat.

A trigger akkor lép működésbe, amikor a táblában valamelyik esemény a három közül megtörtént. Minden sorra alkalmazom, hogy csoportos műveleteknél is működjön. Beépített logikai változókkal vizsgálom a műveleteket, és annak megfelelően hajtom végre a naplózást. Az állapotok lekérdezését a (:NEW és az :OLD) tulajdonságokkal végeztem.

A NAPLO tábla részére is itt végzem az ID létrehozását, melyben az NVL függvényt használtam, hogy akkor is működjön, ha még nincs a NAPLO táblában semmi, és NULL értéket adna. Null esetén a Null-t 0-ra cseréli, így a következő használható ID az 1 lesz.

--6: Eseménynaplózás

BEGIN

EXECUTE IMMEDIATE

'CREATE TABLE NAPLO (

id NUMBER PRIMARY KEY,

muvelet VARCHAR2(20) NOT NULL,

datum DATE DEFAULT SYSDATE NOT NULL,

reszletek VARCHAR2(200)

);

EXECUTE IMMEDIATE

'CREATE OR REPLACE TRIGGER naplo AFTER INSERT OR UPDATE OR DELETE ON UGYFEL FOR EACH ROW

DECLARE

muvelet VARCHAR2(20);

reszletek VARCHAR2(200);

new_id NUMBER;

BEGIN

IF INSERTING THEN

muvelet := 'BESZÚRÁS';

reszletek := 'Új ügyfél: ' || :NEW.id;

ELSIF UPDATING THEN

muvelet := 'MÓDOSÍTÁS';

reszletek := 'Módosított ügyfél: ' || :OLD.id;

ELSIF DELETING THEN

muvelet := 'TÖRLÉS';

reszletek := 'Törölt ügyfél: ' || :OLD.id;

END IF;

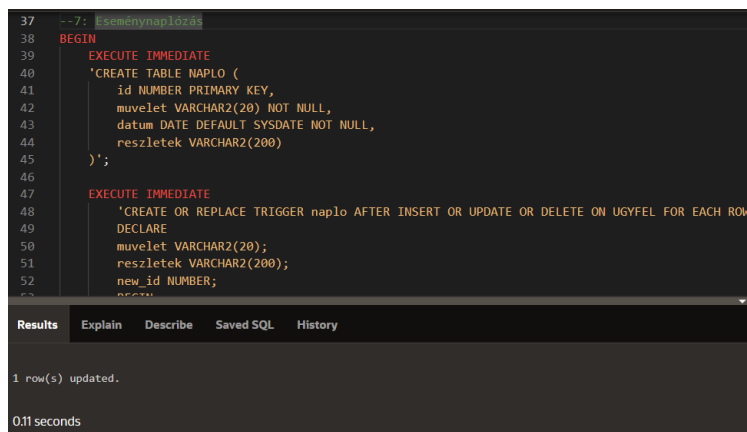
SELECT NVL(MAX(id), 0) + 1 INTO new_id FROM NAPLO;

INSERT INTO NAPLO (id, muvelet, datum, reszletek)

VALUES (new_id, muvelet, SYSDATE, reszletek);

END;';

END;



```
37 --7: Eseménynaplózás
38 BEGIN
39 EXECUTE IMMEDIATE
40 'CREATE TABLE NAPLO (
41 id NUMBER PRIMARY KEY,
42 muvelet VARCHAR2(20) NOT NULL,
43 datum DATE DEFAULT SYSDATE NOT NULL,
44 reszletek VARCHAR2(200)
45 );
46
47 EXECUTE IMMEDIATE
48 'CREATE OR REPLACE TRIGGER naplo AFTER INSERT OR UPDATE OR DELETE ON UGYFEL FOR EACH ROW
49 DECLARE
50 muvelet VARCHAR2(20);
51 reszletek VARCHAR2(200);
52 new_id NUMBER;
53 BEGIN
54 IF INSERTING THEN
55 muvelet := 'BESZÚRÁS';
56 reszletek := 'Új ügyfél: ' || :NEW.id;
57
58 ELSIF UPDATING THEN
59 muvelet := 'MÓDOSÍTÁS';
60 reszletek := 'Módosított ügyfél: ' || :OLD.id;
61
62 ELSIF DELETING THEN
63 muvelet := 'TÖRLÉS';
64 reszletek := 'Törölt ügyfél: ' || :OLD.id;
65 END IF;
66
67 SELECT NVL(MAX(id), 0) + 1 INTO new_id FROM NAPLO;
68
69 INSERT INTO NAPLO (id, muvelet, datum, reszletek)
70 VALUES (new_id, muvelet, SYSDATE, reszletek);
71 END;';
72 END;
```

Results Explain Describe Saved SQL History

1 row(s) updated.

0.11 seconds

8.)Trigger készítése kulcs érték automatikus megadására

A feladat egy olyan adatbázis-szintű trigger megvalósítása volt, amely képes az elsődleges kulcs (ID) automatikus generálására az UGYFEL táblába való beszúráskor. A trigger csak akkor aktiválódik, ha a felhasználó nem ad meg ID-t. Ha nincs megadva manuálisan ID, akkor a trigger megkeresi a legnagyobb ID-t, hozzáad egyet, és ez lesz az új adatsor egyedi azonosítója.

```
--8: ID generálás  
CREATE OR REPLACE TRIGGER ugyfel_id BEFORE INSERT ON UGYFEL FOR EACH ROW  
BEGIN  
    IF :NEW.id IS NULL THEN  
        SELECT NVL(MAX(id), 0) + 1 INTO :NEW.id FROM UGYFEL;  
    END IF;  
END;
```

```
28  --8: ID generálás  
29  CREATE OR REPLACE TRIGGER ugyfel_id BEFORE INSERT ON UGYFEL FOR EACH ROW  
30  BEGIN  
31      IF :NEW.id IS NULL THEN  
32          SELECT NVL(MAX(id), 0) + 1 INTO :NEW.id FROM UGYFEL;  
33      END IF;  
34  END;  
35
```

Results	Explain	Describe	Saved SQL	History
---------	---------	----------	-----------	---------

Trigger created.

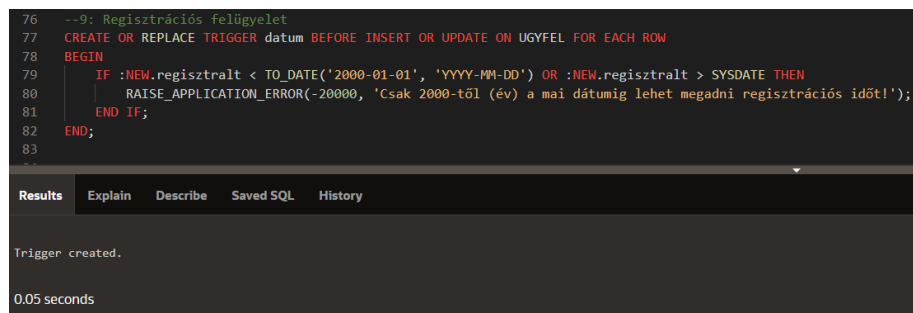
0.03 seconds

9.)Trigger készítése a módosítások kontrollálására.

A feladat egy olyan trigger megírása volt, amely ellenőrzi a felhasználó által bevitt vagy módosított adatokat és korlátozza. Én úgy készítettem el, hogy a regisztrációs dátum csak 2000-től a mai dátumig legyen elfogadott. Pl 2000 legelején nyitott az étterem. És nem lehet a jövőben sem regisztrálni.

Ezt úgy érem el, hogy megvizsgálom a bemenő sorokat, és ha a tiltott intervallumba esik, akkor hibakezeléssel megszakítom a folyamatot. Egyébként pedig végrehajtom.

```
--9: Regisztrációs felügyelet
CREATE OR REPLACE TRIGGER datum BEFORE INSERT OR UPDATE ON UGYFEL FOR EACH ROW
BEGIN
    IF :NEW.regisztralt < TO_DATE('2000-01-01', 'YYYY-MM-DD') OR :NEW.regisztralt > SYSDATE THEN
        RAISE_APPLICATION_ERROR(-20000, 'Csak 2000-től (év) a mai dátumig lehet megadni regisztrációs időt!');
    END IF;
END;
```



```
76 --9: Regisztrációs felügyelet
77 CREATE OR REPLACE TRIGGER datum BEFORE INSERT OR UPDATE ON UGYFEL FOR EACH ROW
78 BEGIN
79     IF :NEW.regisztralt < TO_DATE('2000-01-01', 'YYYY-MM-DD') OR :NEW.regisztralt > SYSDATE THEN
80         RAISE_APPLICATION_ERROR(-20000, 'Csak 2000-től (év) a mai dátumig lehet megadni regisztrációs időt!');
81     END IF;
82 END;
83
```

Results Explain Describe Saved SQL History

Trigger created.

0.05 seconds

TESZTELŐ KÓDOK

--Új ügyfél rögzítése

```
BEGIN
    uj_ugyfel(
        p_nev          => '',
        p_cim          => '',
        p_mobil        => '',
        p_regisztralt => TO_DATE('', 'YYYY-MM-DD'),
        p_szallitas    => --idő
    );
END;
```

--Mobilszám megváltoztatása

```
BEGIN
    ugyfel_mobil(
        p_id    => , --ID
        p_mobil => '' --Új mobilszám
    );
END;
```

--Ügyféltörlés ID alapján

```
BEGIN
    ugyfeltorles_id( p_id => ); --ID
END;
```

--Ügyféltörlés szállítás ideje alapján

```
BEGIN
    ugyfeltorles_szallitas(p_maxido => ); --max elfogadott szállítási idő
END;
```

--Adott azonosítójú ügyfél nevének lekérése

```
SELECT find_nev() AS Keresett_Nev FROM DUAL; --(ID)
```

--Adott azonosítójú ügyfél adatainak lekérése:

```
SELECT ugyfeladatok(1) AS Ugyfel_Adatok FROM DUAL; --(ID)
```

--Átlagos szállítási idő kiszámolása adott településre

```
SELECT szallitasido('') AS Atlagido FROM DUAL; --('településnév vagy irányítószám')
```

--Sima manuális adatbeszúrás

```
BEGIN
    INSERT INTO UGYFEL (nev, cim, mobil, regisztralt, szallitas)
    VALUES ('Varga Judit', '9021 Győr, Baross Gábor út 12.', '3044445566', TO_DATE('2023-09-20', 'YYYY-MM-DD'), 25);
    COMMIT;
END;
```

--Táblák lekérdezése

```
SELECT * FROM UGYFEL ORDER BY id;
```

```
SELECT * FROM NAPLO ORDER BY id;
```

Mélytörlés

BEGIN

BEGIN EXECUTE IMMEDIATE 'DROP TABLE UGYFEL CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN NULL; END;

BEGIN EXECUTE IMMEDIATE 'DROP TABLE NAPLO CASCADE CONSTRAINTS'; EXCEPTION WHEN OTHERS THEN NULL; END;

BEGIN EXECUTE IMMEDIATE 'DROP PROCEDURE uj_ugyfel'; EXCEPTION WHEN OTHERS THEN NULL; END;

BEGIN EXECUTE IMMEDIATE 'DROP PROCEDURE ugyfel_mobil'; EXCEPTION WHEN OTHERS THEN NULL; END;

BEGIN EXECUTE IMMEDIATE 'DROP PROCEDURE ugyfeltorles_szallitas'; EXCEPTION WHEN OTHERS THEN NULL; END;

BEGIN EXECUTE IMMEDIATE 'DROP PROCEDURE ugyfeltorles_id'; EXCEPTION WHEN OTHERS THEN NULL; END;

BEGIN EXECUTE IMMEDIATE 'DROP FUNCTION find_nev'; EXCEPTION WHEN OTHERS THEN NULL; END;

BEGIN EXECUTE IMMEDIATE 'DROP FUNCTION ugyfeladatok'; EXCEPTION WHEN OTHERS THEN NULL; END;

BEGIN EXECUTE IMMEDIATE 'DROP FUNCTION szallitasido'; EXCEPTION WHEN OTHERS THEN NULL; END;

END;